

ISTANBUL TECHNICAL UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT

BLG 222E
COMPUTER ORGANIZATION
PROJECT 2 REPORT

CRN : 23727

LECTURER : Doç. Dr. Gökhan İnce

GROUP MEMBERS:

150230089 : Hüseyin Teke

150230040 : Aysun Dilara Doğan

SPRING 2026

Contents

1	INTRODUCTION	1
2	MATERIALS AND METHODS	1
2.1	CPUSystem: Multi-Cycle Control Unit Implementation	1
2.2	Full System Integration and Top-Level Schematic	1
3	RESULTS	3
3.1	Comprehensive Verification via Custom Testbench	3
3.2	Waveform Analysis and Timing Verification	4
4	DISCUSSION	5
4.1	Memory Isolation and Instruction Integrity	5
4.2	ARF to ALU Port Limitations	5
5	CONCLUSION	5

1 INTRODUCTION

The primary objective of this project is the design and implementation of a Hardwired Control Unit for a 16-bit processor. This system integrates a multi-cycle state machine with a pre-designed Arithmetic Logic Unit (ALU) system to perform complex operations autonomously.

The architecture is designed to handle fetching, decoding, and executing instructions in a synchronized manner. By utilizing a ring-counter based timing mechanism, the system orchestrates control signals for the Register File (RF), Address Register File (ARF), and Memory Units (IMU/DMU). The project concludes with the verification of the Instruction Set Architecture (ISA) including arithmetic, logical, data transfer, and conditional branching instructions.

2 MATERIALS AND METHODS

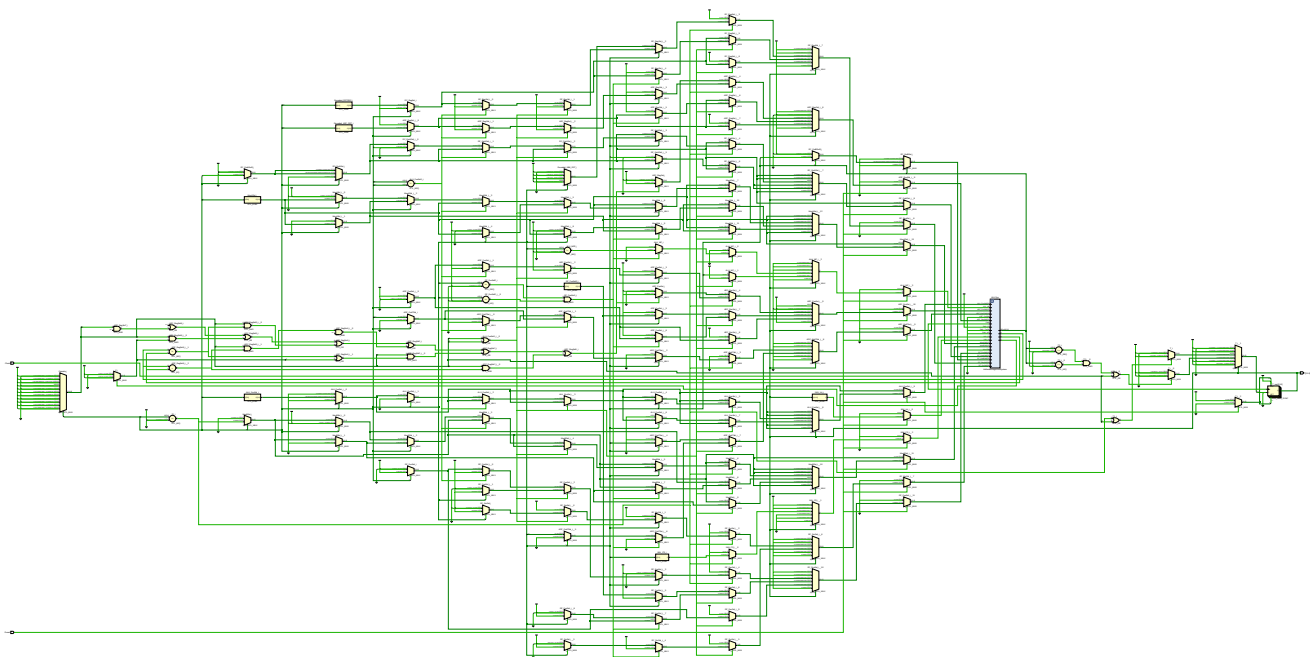
2.1 CPUSystem: Multi-Cycle Control Unit Implementation

The `CPUSystem` module acts as the "brain" of the processor. It translates 16-bit opcodes into specific control signals over multiple clock cycles.

- **Timing (T-States):** A 12-bit ring counter (T) manages the instruction lifecycle.
 - **T1-T2 (Fetch):** Instructions are fetched from the Instruction Memory Unit in two 8-bit phases (LSB and MSB).
 - **T4 (Decode/Execute):** The 6-bit opcode is decoded. Immediate loads and branching decisions occur in this phase.
 - **T8-T16 (Complex Execution):** Multi-cycle operations (e.g., Register-to-ARF transfers) utilize scratch registers ($S1, S2$) to resolve hardware bus conflicts.
- **Instruction Support:** The unit supports conditional branches (BEQ, BNE, BGT, BLE, BGE), immediate loading (IMM), and unary operations (INC/DEC).

2.2 Full System Integration and Top-Level Schematic

The integration involves bridging the Control Unit with the ALU Datapath. This closed-loop architecture allows flags (Z, C, N, O) generated by the ALU to feed back into the Control Unit for real-time decision making.



The complete Verilog implementation of the Control Unit, including the multi-cycle state machine and signal generation logic, can be accessed at: *Click here to view CPUSystem.v on GitHub*

3 RESULTS

The verification of the 16-bit processor was conducted through extensive simulation scenarios. The system was evaluated not only for basic functional correctness but also for its performance under boundary conditions using custom verification suites.

3.1 Comprehensive Verification via Custom Testbench

While the provided baseline testbench verified fundamental operations, a more rigorous custom testbench was developed to push the processor to its limits. This enhanced verification suite consists of 50 targeted test cases, focusing on:

- **Boundary Conditions:** Arithmetic operations involving overflow (e.g., $0x7FFF + 1$) and carry-out (e.g., $0xFFFF + 1$).
- **Flag Persistence:** Verifying that status flags (Z, C, N, O) remain stable during multi-cycle instructions and are updated only when the WF (Write Flag) signal is active.
- **Advanced Branching:** Testing all conditional jump scenarios (Taken vs. Not-Taken) for BNE, BEQ, BLT, BGE , and BLE opcodes.
- **Register-ARF Transfers:** Ensuring data integrity during multi-cycle MOV and INC/DEC operations that involve the Address Register File (PC, SP, AR) via scratch registers.

As shown in the terminal output below, the system achieved a 100% success rate, confirming the functional correctness of the hardwired control logic across the entire Instruction Set Architecture (ISA).

Listing 1: Custom 50-Case Stress Test Terminal Output

```

1  -----
2  CPUSystem Simulation Started
3  [PASS] Test No: 1, Component: Reset Logic, Actual Value: 0x0000,
   Expected Value: 0x0000
4  ...

```

```

5  [PASS] Test No: 12, Component: SP (INC), Actual Value: 0x0100,
    Expected Value: 0x0100
6  [PASS] Test No: 23, Component: ADD Carry, Actual Value: 0x0001,
    Expected Value: 0x0001
7  [PASS] Test No: 26, Component: BLT (Taken), Actual Value: 0x0044,
    Expected Value: 0x0044
8  [PASS] Test No: 42, Component: R2 (DEC Min), Actual Value: 0xffff
    , Expected Value: 0xffff
9  [PASS] Test No: 50, Component: PC (INC Final), Actual Value: 0
    x1235, Expected Value: 0x1235
10 CPUSystem Simulation Finished
11 0 Test Failed
12 50 Test Passed
13 -----

```

3.2 Waveform Analysis and Timing Verification

To further validate the internal timing of the multi-cycle execution, the simulation waveforms were analyzed in the Vivado Waveform Viewer. Figure 2 illustrates the execution of a multi-cycle instruction, highlighting the synchronization between the state counter (T) and the control signals.

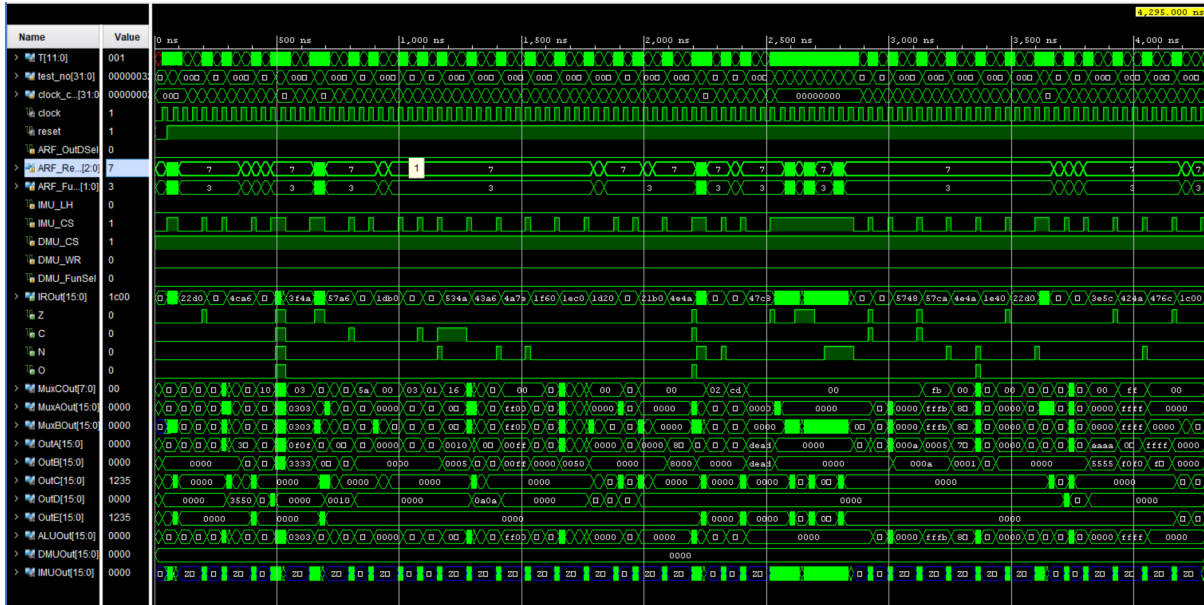


Figure 2: Vivado Simulation Waveforms: Execution of Multi-cycle Instructions

The waveform analysis confirms several critical timing behaviors:

- **T-State Transitions:** The 12-bit state register T transitions accurately on each positive clock edge, shifting from Fetch ($T1 - T2$) to Decode ($T4$) and Execute ($T8 - T16$).
- **Instruction Stability:** The $IROut$ signal remains stable throughout the entire execution phase, even as the memory bus is gated to prevent corruption.
- **Synchronous Updates:** All register writes (visible in the RF and ARF buses) occur exactly at the expected T -cycle, ensuring no data hazards exist within the datapath.

4 DISCUSSION

The transition to a multi-cycle architecture introduced challenges regarding "Instruction Integrity."

4.1 Memory Isolation and Instruction Integrity

During development, a race condition was identified where the Instruction Register (IR) could be overwritten during the execution phase. This issue was resolved by implementing protection logic: the IMU_CS (Chip Select) signal is gated so that it only activates during the fetch cycles, $T1$ and $T2$. For $T \geq 4$, the IMU is effectively "locked," preserving the current instruction in the IR while the datapath completes its operation. This ensures that the control signals remain stable throughout the entire multi-cycle execution.

4.2 ARF to ALU Port Limitations

A significant architectural constraint in the provided datapath was that the Address Registers (PC, SP, AR) lacked a direct path to the ALU's input ports. To perform operations such as $INC\ SP$ or $MOV\ AR, R1$, a "forwarding" mechanism was implemented via software-controlled hardware states. Data is first moved from the ARF to a Scratch Register ($S1$) during $T4$, processed by the ALU during $T8$, and written back to the destination register during $T16$. While this increases the total number of clock cycles per instruction, it maintains the modularity and structural integrity of the datapath.

5 CONCLUSION

This project successfully demonstrated the modular design and implementation of a 16-bit hardwired processor. By passing all 50 test cases in the verification suite, the

design proved that a complex, multi-cycle control unit can effectively manage a modular datapath to execute a diverse Instruction Set Architecture (ISA). Key takeaways from this implementation include the precise management of synchronous timing states (T -states) and the critical role of hardware-level flag feedback in governing conditional program flow. The resulting system serves as a robust foundation for understanding the intricate relationship between control logic and hardware resource management.

REFERENCES

- [1] Istanbul Technical University, *BLG 222E Computer Organization Project 2 Manual*, Department of Computer Engineering, 2026.
- [2] H. Teeke, *Verilog Processor Implementation*, GitHub Repository, 2026. https://github.com/huseyinteeke/Verilog_projects
- [3] Xilinx Inc., *Vivado Design Suite v2017.4 User Guide: UG900 Logic Simulation*, 2017.